HabitPulse: AI-Powered Cross-Habit Intelligence

Master Technical Specification & Step-by-Step Implementation Blueprint

Document Version: 1.0.0	**Target Architecture:** Django 4.2 LTS + PostgreSQL 16 (pgvector)
Status: Production Ready for Implementation	**Operational Cost:** $0.00 / month (100% Free Stack)
Target Directory: /habits	**AI Engine:** Gemini 1.5 Flash (Free Tier)

1. Executive Summary & The Six-Stage Behavioral Loop

Current habit tracking applications operate as **isolated checkbox recorders**. They record whether a user completed 'Gym' or 'Meditation' as independent rows, ignoring that human habits form an interconnected biological and psychological ecosystem. A poor night of sleep directly reduces gym stamina; inadequate hydration creates afternoon fatigue that destroys evening reading habits.

HabitPulse implements the complete six-stage behavioral loop:

Stage	Core Objective	Technical Mechanism
1. Track	Capture micro-metrics and context	Duration, value, time, location, 1-tap energy slider
2. Understand	Recognize daily human context	Energy rating (1-5), stress (Low/Med/High), sleep hours
3. Connect	Identify relationships across habits	Lagged correlation matrix (Day t-1 impact on Day t)
4. Analyze	Eliminate spurious correlations	Domain-constrained hypothesis priors (biological rules)
5. Recommend	Prescribe one high-leverage lever	Grounded RAG synthesis via Google Gemini 1.5 Flash
6. Improve	Sustainable lifestyle progression	Habit stacking, progressive overload, micro-downgrades

2. The Two-Tier Architecture: Zero Hallucination Guarantee

A critical pitfall in AI applications is passing raw logs directly to an LLM and asking it to 'find relationships'. This leads to hallucinations, false causality, and unscientific claims. HabitPulse employs a strict two-tier architecture:

Architecture Tier	Engine & Responsibility	Guarantees & Safeguards

Tier 1: Deterministic Math	Python math & statistical engine running inside Django backend. Calculates Pearson/Spearman correlation coefficients (r), p-values, and co-occurrence percentages across 14-30 day rolling windows.	100% deterministic, 0% hallucination. Ignores sample sizes N < 7. Evaluates temporal precedence (cause must precede effect).
Tier 2: Generative RAG Interpreter	Google Gemini 1.5 Flash Free Tier grounded with behavioral science knowledge chunks from PostgreSQL pgvector.	Strict JSON output schema. Takes mathematically proven facts from Tier 1 and translates them into empathetic, motivating coaching advice.

3. New Data Models & Schema Specifications

All additions are strictly additive, non-destructive, and respect multi-tenant workspace isolation:

Model Name	File Location	Key Fields & Types	Purpose
DailyUserCheckin	habits/models_intelligence.py	user (FK), tenant (FK), checkin_date (Date), energy_level (1-5), stress_level (LOW/MED/HIGH), sleep_hours (Decimal)	Captures daily context in 3 seconds without survey burnout.
HabitCrossCorrelation	habits/models_intelligence.py	primary_habit (FK), secondary_habit (FK), relationship_type (SYNERGY/FRICTION/LAG_RECOVERY), coefficient (Decimal), confidence (Decimal)	Stores verified mathematical correlations between habit pairs.
KnowledgeChunk	habits/models_intelligence.py	category (Char), title (Char), content (Text), embedding (VectorField 768-dim), source_attribution (Char)	Stores curated behavioral science literature for RAG retrieval.
AICoachInsight	habits/models_intelligence.py	user (FK), tenant (FK), insight_type (WARNING/REINFORCE/RECOMMEND), title (Char), body (Text), actionable_step (Char), is_pinned (Bool)	Stores personalized coaching insights surfaced on user dashboard.

4. Step-by-Step Implementation Roadmap (Handoff Instructions)

The executing agent/developer should implement this system in four consecutive, test-verified phases:

Phase 1: Context Layer & Micro-Checkin UI (Immediate Start)

Step 1.1: Create habits/models_intelligence.py with DailyUserCheckin and AICoachInsight models.

Step 1.2: Import intelligence models into habits/models.py and habits/admin.py.

Step 1.3: Generate additive migration: python manage.py makemigrations habits and run python manage.py migrate.

Step 1.4: Implement record_daily_checkin(user, tenant, energy, stress, sleep, note) in habits/services_intelligence.py.

Step 1.5: Add 1-tap energy slider & stress pills component to templates/dashboard/dashboard.html with AJAX submission.

Step 1.6: Write comprehensive tests in tests/test_daily_checkin.py (verifying daily uniqueness, tenant scoping, validation).

Phase 2: Mathematical Correlation Engine

Step 2.1: Create CrossHabitAnalyticsService in habits/services_correlation.py.

Step 2.2: Implement build_daily_behavior_matrix(user, tenant, lookback_days=30) aggregating completion status, durations, and energy.

Step 2.3: Implement compute_lagged_correlations() evaluating Day t-1 Sleep/Water against Day t Gym/Focus habit duration.

Step 2.4: Implement Domain Prior Filter rejecting spurious non-biological correlations (e.g. ignoring flossing vs reading).

Step 2.5: Write unit tests in tests/test_cross_habit_correlations.py with synthetic multi-day behavioral datasets.

Phase 3: Zero-Cost RAG Vector Engine ($0.00 Cost)

Step 3.1: Update docker-compose.yml database image to pgvector/pgvector:pg16 and restart containers.

Step 3.2: Enable vector extension in PostgreSQL: CREATE EXTENSION IF NOT EXISTS vector;.

Step 3.3: Create KnowledgeChunk model with 768-dimensional VectorField.

Step 3.4: Build seed management command habits/management/commands/seed_rag_knowledge.py with 50+ curated science chunks.

Step 3.5: Implement retrieve_relevant_coaching_context(query_text, top_k=3) using cosine similarity search in Django ORM.

Phase 4: AI Personal Growth Coach & Weekly Digest

Step 4.1: Implement generate_weekly_coach_digest(user, tenant) in habits/services_coach.py using Gemini 1.5 Flash.

Step 4.2: Create dashboard Coach Drawer UI (templates/dashboard/includes/ai_coach_drawer.html) with chat Q&A.;

Step 4.3: Add API endpoints /api/coach/chat/ and /api/coach/insights/vote/ for interactive feedback.

Step 4.4: Write automated view tests in tests/test_ai_coach_views.py with mocked LLM calls for lightning-fast CI/CD.

5. Domain Prior Filter & Anti-Hallucination Guardrails

To ensure the AI never embarrasses the product with silly correlations, statistical findings are filtered against a verified scientific prior dictionary:

Prior Category	Permitted Hypothesis Pathway	Behavioral Science Grounding
Sleep -> Fitness	Sleep duration/latency -> Workout completion & duration	Sleep deficit reduces muscle glycogen synthesis and raises perceived exertion.

Hydration -> Stamina	Daily water logged -> Aerobic duration & evening energy	Even 2% dehydration impairs aerobic endurance by 15% and causes headache fatigue.
Stress -> Habit Volume	High work stress day -> Failure of complex habits	BJ Fogg Model: When cognitive load spikes, ability collapses; micro-downgrades required.
Habit Stacking	Anchor habit (streak > 21) -> New habit execution	Existing automatic neural pathways act as reliable cues for nascent routines.

6. Zero-Cost Operational Budget Audit ($0.00 / Month)

Every single component has been engineered to operate completely free of charge, supporting up to 500 active users before any cloud tier upgrades are needed:

Service / Component	Technology Used	Free Tier Allowance	Monthly Cost
Vector Database	pgvector on PostgreSQL 16 (Docker)	Unlimited local/container storage	**$0.00**
LLM Reasoning & Chat	Google Gemini 1.5 Flash	1,500 requests/day, 15 RPM, 1M context	**$0.00**
Text Embeddings	Gemini text-embedding-004	1,500 requests/minute	**$0.00**
Correlation Math	Pure Python & PostgreSQL SQL	Local CPU / zero external APIs	**$0.00**
Total Monthly Cost	**Entire AI Stack**	**500 Daily Active Users**	**$0.00 / month**

7. Quality Checklist & Definition of Done

The executing agent/developer must ensure all items pass before closing any implementation task:

- [ ] Zero database data loss: existing users, tenants, and habits remain 100% intact.
- [ ] Multi-tenant isolation verified: checkins and insights are strictly scoped to request.tenant.
- [ ] Statistical correlations tested with synthetic datasets covering positive, negative, and null scenarios.
- [ ] External Gemini API calls wrapped in robust try/except with deterministic fallback templates.
- [ ] All automated unit and view tests pass cleanly with pytest (exit code 0).
- [ ] UI renders cleanly on mobile viewports (360px) and desktop (1440px).